

> Question 1: Exception Handling - Robust Calculator

↳ 1 cell hidden

> Question 2: File Operations - Text File Manager

↳ 1 cell hidden

> Question 3: CSV File Processing with Error Handling

↳ 2 cells hidden

> Question 4: JSON Configuration Manager

🔍 ↳ 1 cell hidden

✓ Question 5: Logging System with File Management

```
import os
from datetime import datetime

LOG_FILE = "app.log"
MAX_LOG_SIZE = 1000 # bytes

def rotate_log(log_file):

    try:

        if (
            os.path.exists(log_file)
            and os.path.getsize(log_file) > MAX_LOG_SIZE
        ):

            timestamp = datetime.now().strftime(
                "%Y-%m-%d_%H-%M-%S"
            )

            archive_name = (
                f"app_{timestamp}.log"
            )

            os.rename(
                log_file,
                archive_name
            )

            print(
                f"Log file size exceeded. Rotating..."
            )

            print(
                f"Old log archived as: "
                f"{archive_name}"
            )

        except Exception as e:

            print(
                f"Log rotation failed: {e}"
            )

def log_message(
    level,
```

```
message,
log_file=LOG_FILE
):

    allowed_levels = [
        "INFO",
        "WARNING",
        "ERROR",
        "DEBUG"
    ]

    if level not in allowed_levels:

        print(
            f"Invalid log level: {level}"
        )

        return

    rotate_log(log_file)

    timestamp = datetime.now().strftime(
        "%Y-%m-%d %H:%M:%S"
    )

    log_entry = (
        f"[{timestamp}] "
        f"[{level}] "
        f"{message}\n"
    )

    try:

        with open(
            log_file,
            "a"
        ) as file:

            file.write(log_entry)

            print(log_entry.strip())

    except PermissionError:

        print(
            "Permission denied."
        )

        print(
            "Fallback Console Log:"
        )

        print(log_entry.strip())

    except OSError as e:

        print(
            f"File Error: {e}"
        )

        print(
            "Fallback Console Log:"
        )

        print(log_entry.strip())

    except Exception as e:

        print(
            f"Unexpected Error: {e}"
        )

        print(
            "Fallback Console Log:"
        )

        print(log_entry.strip())
```

```

def read_recent_logs(
    log_file,
    count=5
):
    try:
        with open(
            log_file,
            "r"
        ) as file:
            lines = file.readlines()

            print(
                f"\nReading recent logs "
                f"(last {count} entries):"
            )

            for line in lines[-count:]:
                print(line.strip())

    except FileNotFoundError:
        print(
            "Log file not found."
        )

# -----
# Filter By Level
# -----

def filter_logs(
    log_file,
    level
):
    try:
        print(
            f"\nFiltering "
            f"{level} logs:"
        )

        with open(
            log_file,
            "r"
        ) as file:
            for line in file:
                if f"[{level}]" in line:
                    print(
                        line.strip()
                    )

    except FileNotFoundError:
        print(
            "Log file not found."
        )

def search_logs(
    log_file,
    keyword
):
    try:
        print(
            f"\nSearching logs "
            f"for keyword: "
            f"{keyword}"
        )

```

```
        with open(
            log_file,
            "r"
        ) as file:

            found = False

            for line in file:

                if keyword.lower() in (
                    line.lower()
                ):
                    print(
                        line.strip()
                    )
                    found = True

            if not found:

                print(
                    "No matching logs found."
                )

        except FileNotFoundError:

            print(
                "Log file not found."
            )

    print(
        "=== Logging System ==="
    )

    print(
        "\nLogging messages..."
    )

    log_message(
        "INFO",
        "Application started"
    )

    log_message(
        "WARNING",
        "Low memory detected"
    )

    log_message(
        "ERROR",
        "Failed to connect to database"
    )

    log_message(
        "DEBUG",
        "Processing user request"
    )

    read_recent_logs(
        LOG_FILE
    )

    filter_logs(
        LOG_FILE,
        "ERROR"
    )

    search_logs(
        LOG_FILE,
        "database"
    )

    print(
        "\nLogging system demonstration complete."
    )
```

```

=== Logging System ===

Logging messages...
[2026-06-09 10:11:19] [INFO] Application started
[2026-06-09 10:11:19] [WARNING] Low memory detected
[2026-06-09 10:11:19] [ERROR] Failed to connect to database
[2026-06-09 10:11:19] [DEBUG] Processing user request

Reading recent logs (last 5 entries):
[2026-06-09 10:11:19] [INFO] Application started
[2026-06-09 10:11:19] [WARNING] Low memory detected
[2026-06-09 10:11:19] [ERROR] Failed to connect to database
[2026-06-09 10:11:19] [DEBUG] Processing user request

Filtering ERROR logs:
[2026-06-09 10:11:19] [ERROR] Failed to connect to database

Searching logs for keyword: database
[2026-06-09 10:11:19] [ERROR] Failed to connect to database

Logging system demonstration complete.

```

Question 6: Data Backup System

```

import os
import shutil
import hashlib

from datetime import datetime

SOURCE_FILE = "data.txt"

with open(SOURCE_FILE, "w") as file:

    file.write(
        "This is sample data for backup.\n"
    )

    file.write(
        "Backup verification test.\n"
    )

print(
    f"Sample file '{SOURCE_FILE}' created.\n"
)

def calculate_checksum(file_path):

    md5 = hashlib.md5()

    with open(file_path, "rb") as file:

        while True:

            chunk = file.read(4096)

            if not chunk:
                break

            md5.update(chunk)

    return md5.hexdigest()

def cleanup_old_backups(
    backup_root,
    keep=10
):

    all_backups = []

    for root, dirs, files in os.walk(
        backup_root
    ):

        for file in files:

```

```

        path = os.path.join(
            root,
            file
        )

        all_backups.append(path)

    all_backups.sort(
        key=os.path.getmtime,
        reverse=True
    )

    old_files = all_backups[keep:]

    removed = 0

    for file in old_files:

        os.remove(file)

        removed += 1

    print(
        f"Kept: {min(len(all_backups), keep)} "
        f"most recent backups"
    )

    print(
        f"Removed: {removed} old backups"
    )

def backup_file(
    source_file
):
    try:

        print(
            "=== Data Backup System ==="
        )

        print(
            f"\nBacking up "
            f"'{source_file}'..."
        )

        if not os.path.exists(
            source_file
        ):

            raise FileNotFoundError(
                f"Source file "
                f"'{source_file}' "
                f"not found."
            )

        today = datetime.now().strftime(
            "%Y-%m-%d"
        )

        timestamp = datetime.now().strftime(
            "%Y-%m-%d_%H-%M-%S"
        )

        backup_dir = os.path.join(
            "backups",
            today
        )

        os.makedirs(
            backup_dir,
            exist_ok=True
        )

        base_name = os.path.splitext(
            os.path.basename(

```

```
        source_file
    )
)[0]

extension = os.path.splitext(
    source_file
)[1]

backup_file_name = (
    f"{base_name}_"
    f"{timestamp}"
    f"{extension}"
)

destination = os.path.join(
    backup_dir,
    backup_file_name
)

print(
    f"Destination: "
    f"{destination}"
)

shutil.copy2(
    source_file,
    destination
)

print(
    "Backup created successfully."
)

print(
    "\nVerifying backup..."
)

if not os.path.exists(
    destination
):
    raise IOError(
        "Backup verification failed."
    )

source_size = os.path.getsize(
    source_file
)

backup_size = os.path.getsize(
    destination
)

if source_size != backup_size:
    raise IOError(
        "File size mismatch."
    )

source_hash = calculate_checksum(
    source_file
)

backup_hash = calculate_checksum(
    destination
)

if source_hash != backup_hash:
    raise IOError(
        "Checksum mismatch."
    )

print(
    f"Backup verified "
    f"(size: {backup_size:,} bytes)"
```

```

    )

    print(
        "\nCleaning old backups..."
    )

    cleanup_old_backups(
        "backups",
        keep=10
    )

except FileNotFoundError as e:

    print(
        f"Error: {e}"
    )

except PermissionError:

    print(
        "Error: Permission denied."
    )

except OSError as e:

    print(
        f"File System Error: {e}"
    )

except IOError as e:

    print(
        f"Verification Error: {e}"
    )

except Exception as e:

    print(
        f"Unexpected Error: {e}"
    )

backup_file(
    SOURCE_FILE
)

print(
    "\n=== Error Test ==="
)

backup_file(
    "nonexistent.txt"
)

```

Sample file 'data.txt' created.

=== Data Backup System ===

Backing up 'data.txt'...

Destination: backups/2026-06-09/data_2026-06-09_10-14-41.txt

Backup created successfully.

Verifying backup...

Backup verified (size: 58 bytes)

Cleaning old backups...

Kept: 1 most recent backups

Removed: 0 old backups

=== Error Test ===

=== Data Backup System ===

Backing up 'nonexistent.txt'...

Error: Source file 'nonexistent.txt' not found.